CS117 Project

Vincent Wang

June 2026

1 Project Overview

The goal of this project was to reconstruct a complete 3D mesh of an object using a structured light scanning system. The system consisted of two calibrated cameras and a projector that displayed binary structured light patterns onto the object. By decoding the projected patterns in both camera views, corresponding pixels could be identified and triangulated into 3D points. Each individual scan, or "grab," produced a partial reconstruction of the object from one viewpoint. These partial point clouds and meshes were then aligned together to form a more complete 3D model.

The overall pipeline consisted of several major stages: camera calibration, structured light decoding, triangulation, point cloud cleanup, correspondence point alignment, mesh smoothing, and final mesh processing in MeshLab.

1.1 System Overview

First, I will summarize the reconstruction pipeline. The reconstruction pipeline consists of six major stages:

1. Camera calibration
2. Structured light decoding
3. Stereo triangulation
4. Point cloud cleanup
5. Multi-view scan alignment
6. Mesh Generation
7. Surface reconstruction and mesh refinement

1.2 Data Description

Data Description The dataset consists of calibration images and structured-light scans collected using a stereo camera setup and a projector. For camera calibration, each camera captured 12 checkerboard images with the checkerboard positioned at a variety of orientations and viewpoints.

For each object scan, both cameras captured a sequence of structured-light images projected onto the scene. A total of 40 structured-light images were collected per camera for each scan, consisting of 20 images encoding vertical Gray-code patterns and 20 images encoding horizontal Gray-code patterns.

In addition to the structured-light images, each camera captured two color images per scan: one image of the scene without the object and one image with the object present.

The David statue dataset contains scans from five distinct viewpoints, denoted as grabs 0 through 4. While the viewpoint of the object changed between scans, the relative positions of the cameras and projector remained fixed throughout the data collection process.

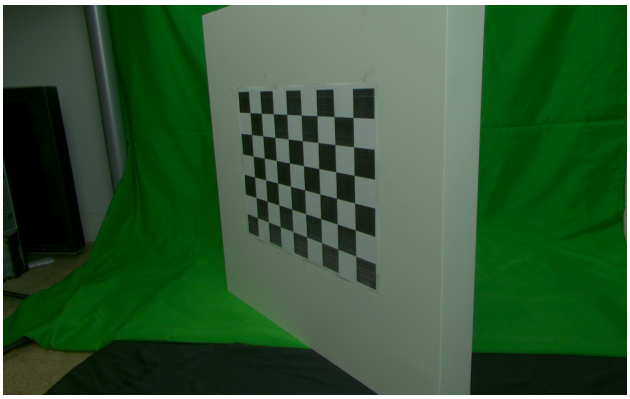


Figure 1: Example Calibration Image

2 Algorithms

2.1 Calibration

The first step was to calibrate each camera independently. Because we did not know any prior properties of the cameras, including focal length, principle point, and rotation and translation from the world center, we need some way to extract these parameters.

Camera calibration was performed independently for both cameras using OpenCV's checkerboard calibration pipeline. A set of calibration images containing an 8×6 checkerboard pattern with known square spacing (2.8 units) was collected for each camera, as shown in Figure 1. For every image, the algorithm detected the checkerboard corners using `cv2.findChessboardCorners()`, producing a set of 2D image coordinates corresponding to known 3D checkerboard coordinates.

These 2D–3D correspondences from all calibration images were then provided to `cv2.calibrateCamera()`, which estimates the intrinsic camera parameters, including focal lengths (f_x, f_y) and principal point coordinates (c_x, c_y). Furthermore, the `cv2.calibrateCamera()` function provided rotation and translation vectors which describe the pose of the checkerboard with respect to the camera coordinate system.

Using the rotation vector for the checkerboard calibration image, it was passed into the `cv2.Rodrigues()` function and converted to a rotation matrix using Rodrigues' formula. Since OpenCV returns the transformation from the checkerboard coordinate frame to the camera frame, this transformation was inverted to obtain the camera pose in the checkerboard world coordinate system. Specifically, the rotation matrix was transposed and the translation vector was transformed according to

$$R = R_{\text{obj}\to\text{cam}}^T$$

$$t = -R_{\text{obj}\to\text{cam}}^T t_{\text{obj}\to\text{cam}}$$

The resulting intrinsic and extrinsic parameters were stored in `Camera` objects and saved for later use during structured-light reconstruction and triangulation.

The camera matrix has the form

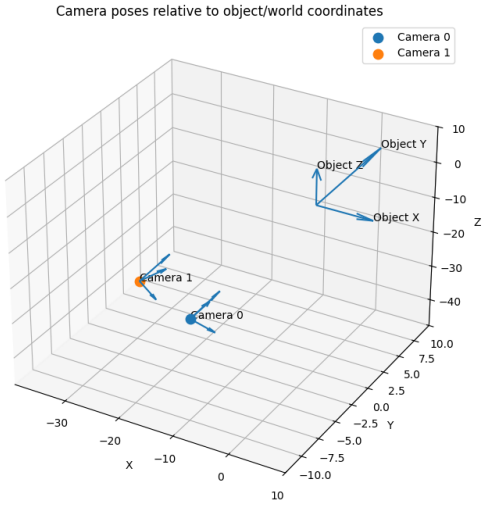
$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

where f_x and f_y are the focal lengths in pixel units, and (c_x, c_y) is the principal point.

2.2 Pixel Reconstruction and Structured Light Decoding

After calibration, structured light decoding was used to find pixel correspondences between the two cameras.

Using structured light, a sequence of black-and-white illumination patterns is projected onto the scene. Each pattern encodes one bit of information using Gray code, where a point on the object is assigned 1 if



it appears bright in a particular pattern and 0 if it appears dark. Since a 10-bit Gray code is used, ten pattern pairs are projected, allowing up to ($2^{10} = 1024$) unique values to be encoded. For each camera pixel, the captured sequence of bright and dark observations forms a 10-bit Gray code. This Gray code is then converted into a standard binary number, producing an integer identifier corresponding to a projector coordinate.

The process is performed twice: once using patterns that encode the horizontal projector coordinate and once using patterns that encode the vertical projector coordinate. As a result, every visible point on the object is assigned a unique projector coordinate ((H,V)). These coordinates are combined into a single unique code, allowing the same physical point to be identified in both camera images. If a pixel in the left image and a pixel in the right image decode to the same projector coordinate, they are assumed to correspond to the same point on the object's surface. This eliminates the difficult stereo correspondence problem and enables dense matching across the entire visible surface. Once corresponding pixels have been identified, the calibrated camera models are used to triangulate their viewing rays and recover the 3D location of the surface point.

The decoding threshold is used to remove pixels whose Gray code values are unreliable. For each Gray code bit, two images are captured: the original projected pattern and its inverse. When the absolute difference between the two images is very small, the pixel intensity is too ambiguous to confidently determine whether the projected pattern was bright or dark at that location. Therefore, we set a specific threshold, called the decoding threshold, which invalidates a given pixel if the difference between pixel intensities is

smaller than the decoding threshold for either the left or right camera image.

Notably, Gray code was used instead of standard binary because adjacent codes differ by only one bit. This reduces large decoding errors near structured light boundaries, eliminating sudden jumps in coordinates.

Furthermore, we compare colored images between the object and the background, to compute a foreground mask. The foreground mask is simply the difference between an image with the object in place, and an image without the object in place. Using this, we are able to determine whether a specific pixel is within the foreground or not. Only pixels within the foreground mask with a difference greater than a specified color threshold were kept.

$$M_{\text{valid}} = M_{\text{decode}} \cap M_{\text{foreground}}.$$

2.3 Bounding Box Pruning

Bounding box pruning was used to remove points outside a manually chosen valid 3D region. This is useful because triangulation can produce outliers far away from the actual object due to decoding mistakes, background pixels, or poor correspondences.

A typical bounding box filter keeps only points satisfying

$$x_{\min} < x < x_{\max}, \quad y_{\min} < y < y_{\max}, \quad z_{\min} < z < z_{\max}.$$

For initial bounding box pruning, we eliminate any obvious outliers throughout the image, however we keep the turntable and the boxes below the object. We will need these points in the following section, when performing correspondence matching.

2.4 Correspondence Points Matching

Each scan produced a separate partial reconstruction. To combine scans, corresponding 3D points had to be identified between different grabs. These correspondences were used to estimate the SVD transformation that maps one scan into the coordinate frame of another scan.

To do this, we load colored images of the model for each scan, and then manually match common points between the two images. Each clicked 2D point was then matched to the nearest reconstructed 2D pixel, allowing the corresponding 3D point to be retrieved.

2.4.1 Correspondence Points with a Single Base Grab

One approach was to choose one grab as the fixed base scan and align every other scan directly to this reference. Initially, I thought this would be the most optimal solution because it avoids accumulating error through a chain of pairwise alignments. In other words, each scan is transformed only once into the base coordinate system, rather than being repeatedly transformed through intermediate scans.

However, this approach can fail, as seen in Figure 2, when a scan does not have enough overlap with the chosen base scan. If two scans view very different sides of the object, the corresponding points are harder to identify accurately, and the resulting transformation can be poor.

This was the exact problem that I ran into when testing this implementation, as no matter which scan I assigned as a base scan, there were always at least one other scan which was not able to be aligned properly, as corresponding points were either impossible to find, or extremely inaccurate.

Therefore, although direct alignment to a base scan reduces accumulated drift, it depends heavily on the base scan having sufficient overlap with every other scan.

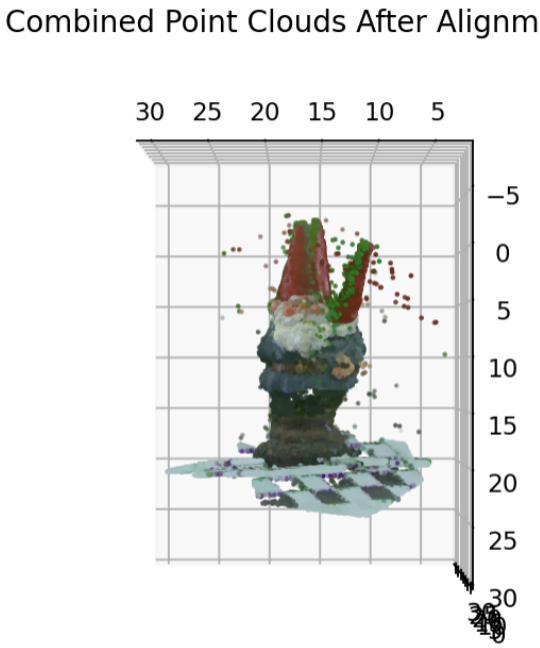


Figure 2: Corresponding with Single Base

2.4.2 Correspondence Points Using the Previous Grab to Match

The second, and better approach was to align each grab to the previous grab instead of always aligning to the base. For example, grab 3 is aligned to grab 2, grab 4 is aligned to grab 3, and so on. The transformations are then chained together to bring all scans into the base coordinate frame.

While this resulted in accumulated error between scans, it actually produced better results as better corresponding points were able to be produced at every point.

Furthermore, I implemented a distance-based filtering step to improve the quality of the manually selected correspondences. For each user-selected point in the left and right images, the nearest reconstructed 2D point was located. Let (p_L) and (p_R) denote the user-selected points in the left and right images, and let $(\hat{p}_L)$ and $(\hat{p}_R)$ denote their nearest reconstructed image points. The Euclidean distances

$$d_L = |p_L - \hat{p}_L|_2$$

and

$$d_R = |p_R - \hat{p}_R|_2$$

were computed for each correspondence pair. A correspondence was retained only if both distances were below a predefined threshold (τ):

$$d_L < \tau \quad \text{and} \quad d_R < \tau.$$

If either distance exceeded the threshold, the correspondence was discarded. This filtering process removed inaccurate point selections caused by user error, sparse reconstruction regions, or mismatches between image features and reconstructed points. As a result, although 5–10 correspondence pairs were typically selected initially, only the most reliable 3–5 correspondences were retained for estimating the SVD transformation. Using a smaller set of highly accurate correspondences generally produced better alignment results than including additional correspondences with large localization errors, as found by Figure 2.4.2

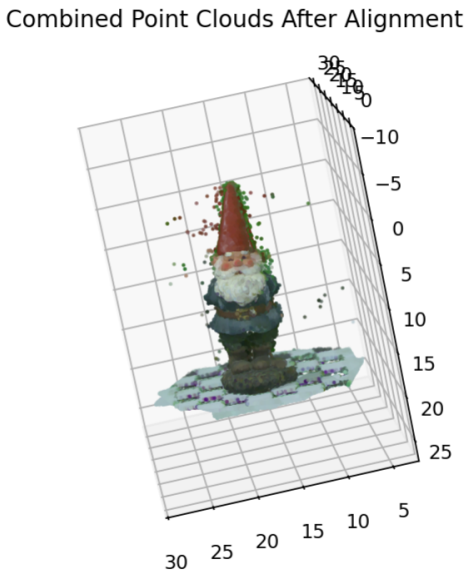


Figure 3: Corresponding with Previous Grab

2.4.3 Correspondence Selection Challenges

Accurate correspondences are critical because all subsequent SVD alignment depends on them.

However, there were significant problems involved with finding best possible corresponding points. Initially, I tried to increase the color and decoding thresholds as high as possible, to eliminate as much extraneous or inaccurate points as possible. However, in doing so, I reduced the amount of points which could be used in the correspondence search. Because of this, I was failing to get at least 3 accurate correspondence points. To solve this, I was forced to increase both the color and decoding thresholds to extract as many possible points as possible.

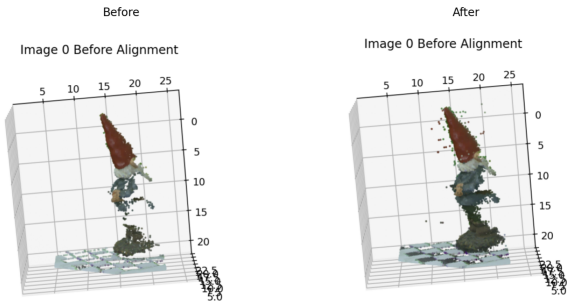


Figure 4: Before and After Threshold Changes

This allowed me to increase the number of corresponding points to select, and allowed me to choose additional points on the checkerboard, which created the most optimal point cloud alignment.

2.5 SVD Alignment

Once corresponding 3D points were selected, SVD was used to compute the rotation and translation between two scans. Given two sets of corresponding points,

$$P = \{p_i\}, \qquad Q = \{q_i\},$$

the goal is to find R and t such that

$$q_i \approx Rp_i + t.$$

The SVD method solves this least-squares problem by first subtracting the centroids of both point sets, computing a covariance matrix, and then extracting the optimal rotation.

This alignment step was essential because each grab was reconstructed in its own local coordinate system. SVD alignment allowed all scans to be combined into one global model.

2.6 ICP Point-to-Point Refinement

After manual correspondence alignment, Iterative Closest Point, or ICP, can be used to refine the registration. While SVD alignment depends only on a small number of manually clicked correspondences, ICP uses many points from the point clouds.

The baseline goal of ICP is to reduce small residual alignment errors by bringing the point cloud closer together overall. I considered using several different numbers of iterations, ranging from 0 to 20. The resulting differences are shown in Figure 2.6

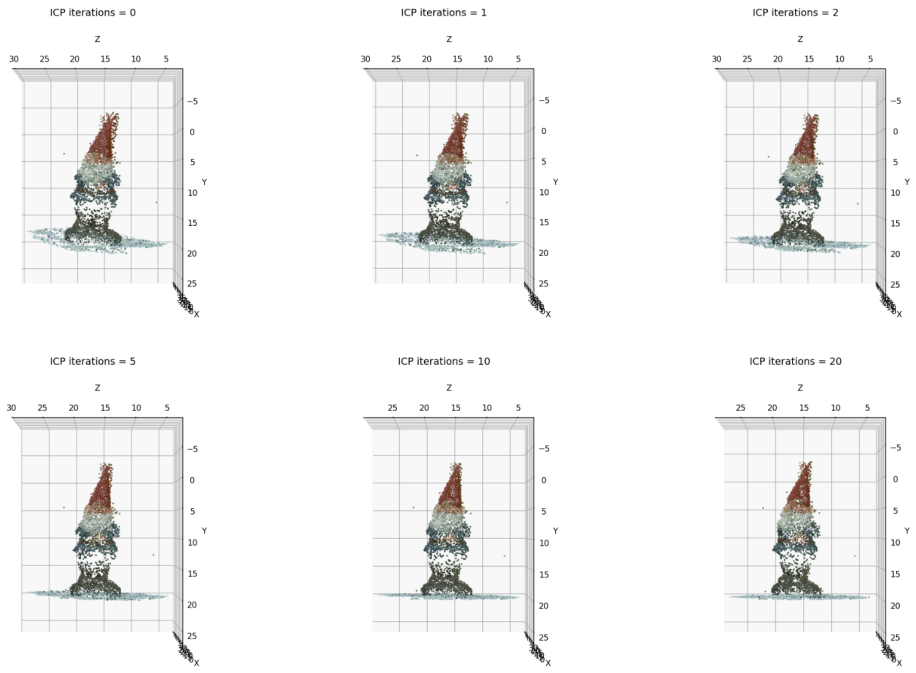


Figure 5: Comparison of Point Meshes for Various ICP Iterations

It can be seen that around 10 and 20 iterations is when ICP works best, and truly results in nice alignments.

2.7 Removing Background Points

By correspondence matching with lower decoding and color thresholds, I was able to get much more accurate correspondence matches. However, this led to a lot more background noise, as seen in Figure 2.4.2. To reduce noise and remove as many background points as possible, I first applied an additional bounding box to the combined point cloud, which removed the checkerboard pattern below the image, as well as any extreme outliers from the mesh.

Removing Green-screen Points However, as seen in Figure 2.4.2, the reduction in color thresholding led to a lot of additional green points from the green-screen background, as seen by Figure 6.

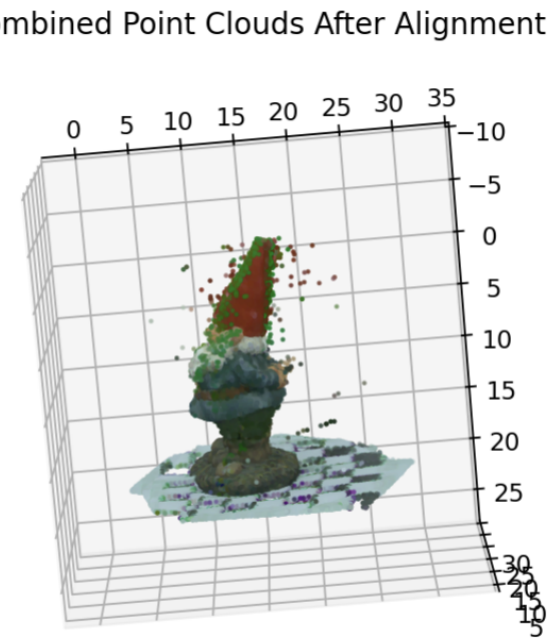


Figure 6: Post Alignment Point Mesh

To remove points belonging to the green-screen background, a color-based mask was applied. Let R_i, G_i, and B_i denote the red, green, and blue color values of point i, and let τ denote the green threshold parameter. Note that $R_i, G_i, B_i \in [0, 1]$. A point was classified as a green-screen point if its green intensity exceeded 0.3 and was greater than both its red and blue intensities by at least τ. Formally, the mask is given by

$$M_i = \neg \left((G_i > 0.3) \ \wedge \ (G_i - R_i > \tau) \ \wedge \ (G_i - B_i > \tau) \right).$$

Only points satisfying M_i are retained. Thus, points whose color is strongly dominated by green are removed. This significantly reduced the amount of obvious green points on the mesh. The results can be seen in Figure 7.

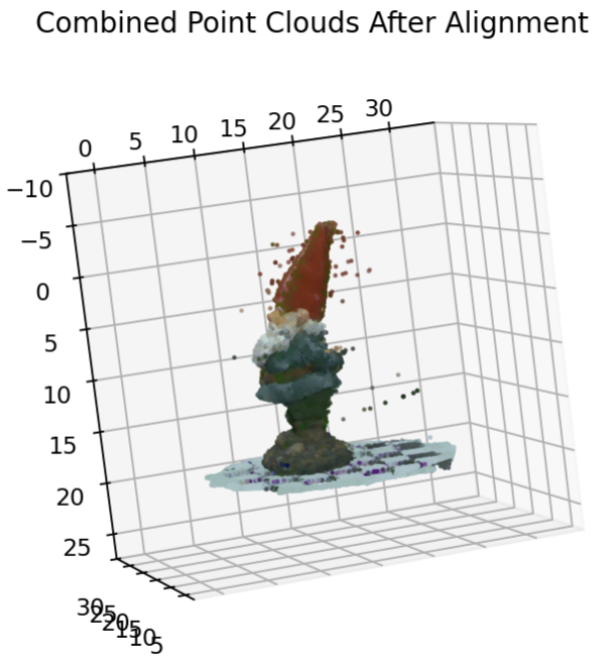


Figure 7: Green Point Thresholding

2.8 Triangulation

After structured light decoding produced pixel correspondences, triangulation was used to recover 3D points. I use the `scipy.spacial.Delaunay()` triangulation method to compute this triangulation.

2.8.1 Triangle Threshold Pruning

After triangulation, some triangles connected points that were very far apart in 3D space. Triangle threshold pruning removes triangles whose edge lengths are too large.

By specifying a maximum allowed edge length, we can remove bad triangles with edge lengths too large to be able to connect close points in 3D space. Typical meshes contain triangles that are extremely small in edge length.

A smaller threshold removes more bad triangles but can create holes. A larger threshold preserves more surface area but may keep incorrect stretched triangles. I found that a threshold of 1.2 was best for removing significantly large triangles, as shown in Figure 8.

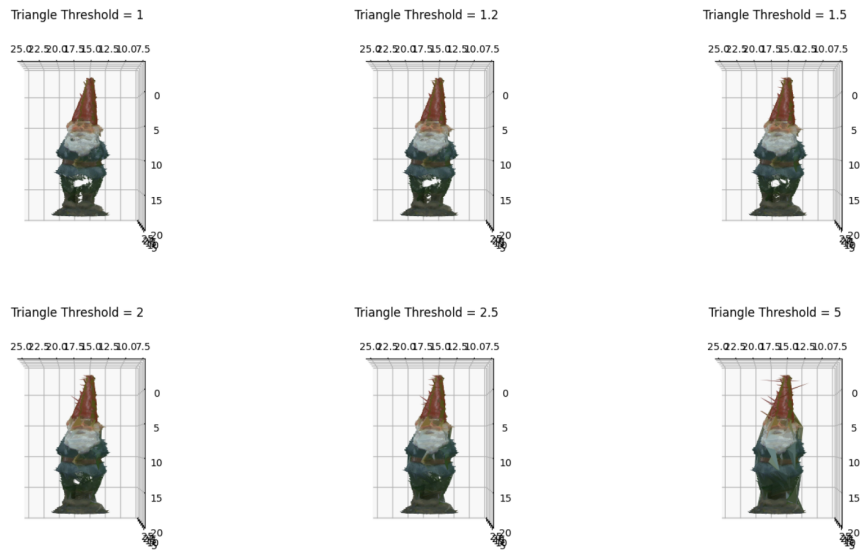


Figure 8: Meshes for Varying Triangle Thresholding

2.9 Mesh Smoothing

Mesh smoothing was used to reduce small surface irregularities after reconstruction and pruning. The implemented smoothing method moves each triangle's vertices slightly toward the triangle centroid.

The update rule is

$$p_i' = p_i + \alpha(c - p_i),$$

where c is the triangle centroid and $\alpha = 0.1$ in this implementation.

The main parameter is the number of smoothing iterations. More smoothing reduces noise, as well as makes the object less spiky around the outside, but can also remove fine details. This can be seen in Figure 9.

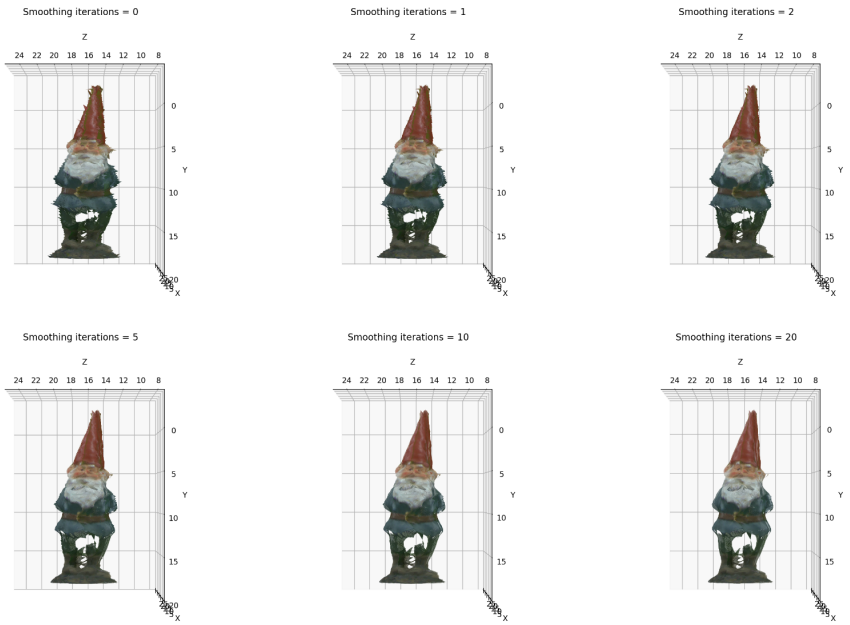


Figure 9: Mesh Smoothing Comparison

2.10 MeshLab Processing

After the scans were aligned and combined, MeshLab was used for final mesh processing. The first step was to flatten or clean the surface to remove remaining irregularities and prepare the point cloud for reconstruction. This was done using the **flatten visible layers** feature. The resulting mesh after this is shown in Figure 10. Then Poisson surface reconstruction was used to generate a continuous watertight mesh from the oriented point cloud.

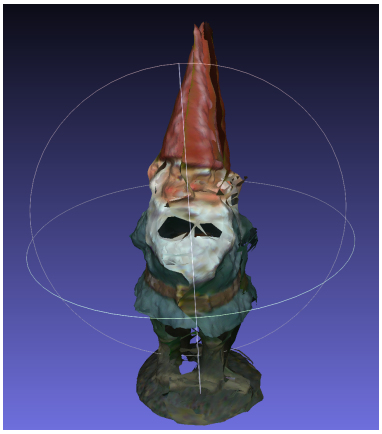


Figure 10: Before Poisson Reconstruction

The baseline goal of MeshLab processing was to convert a noisy combined point cloud into a cleaner final surface.

Using Poisson reconstruction I was able to fill in any small holes that resulted from pruning, and create a smoother complete mesh, as seen in Figure 11.

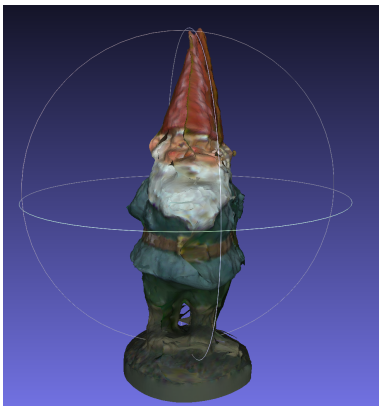


Figure 11: After Poisson Reconstruction

3 Results and Discussion

Several sources of error remained in the reconstruction pipeline.

- Structured light decoding failed to gather many points on the body of the gnome when thresholds were too high, but generated outliers when thresholds were too small.
- Manual correspondence selection is susceptible to human error.

- Triangle pruning created small holes in the mesh, which was filled with much less detail by Poisson reconstruction.
- Too much surface smoothing removed fine geometric detail.

Despite these sources of error, the combination of structured light decoding, SVD alignment, point cloud cleanup, and surface reconstruction produced a fairly visually accurate representation of the scanned object.

Figure 12: Original 3D Model

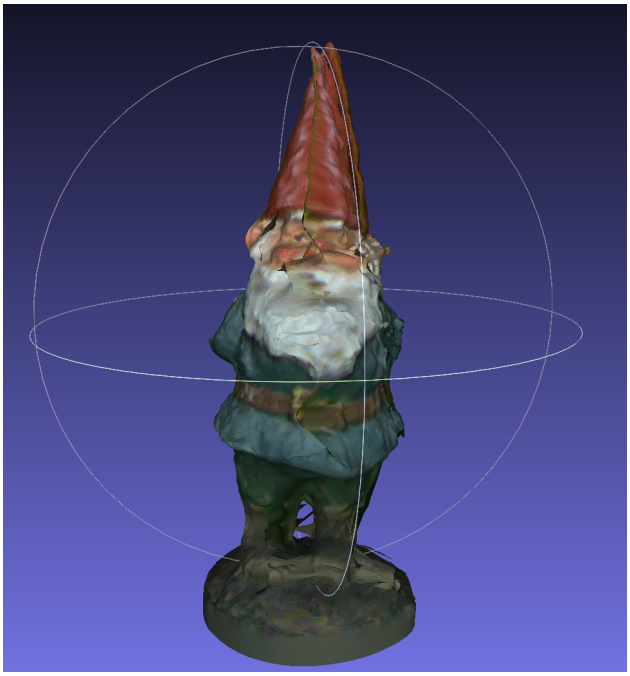


Figure 13: Front View

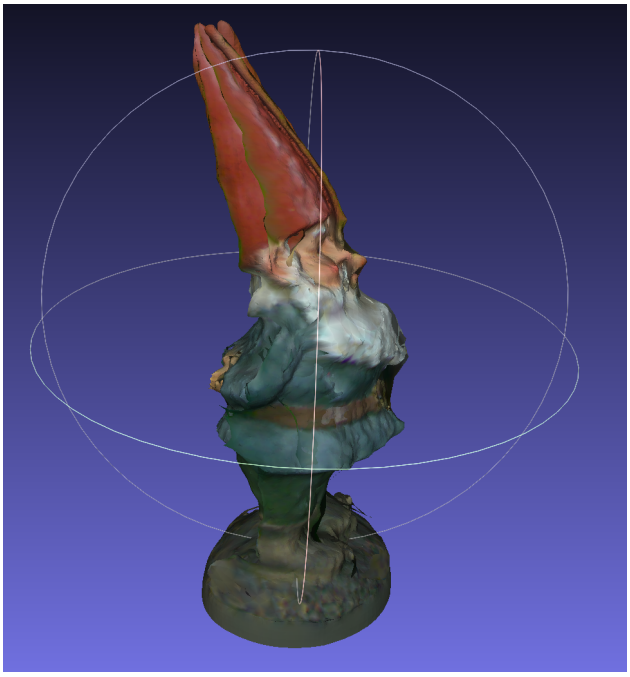


Figure 14: Right Side View

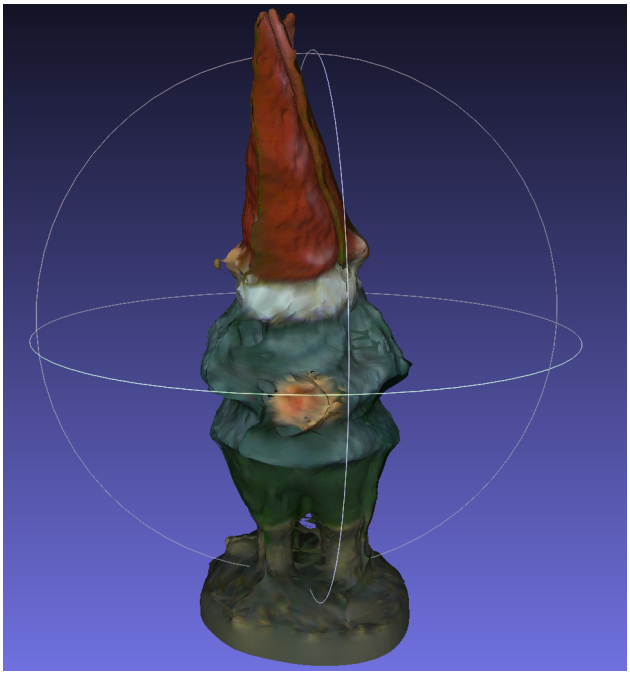


Figure 15: Back View

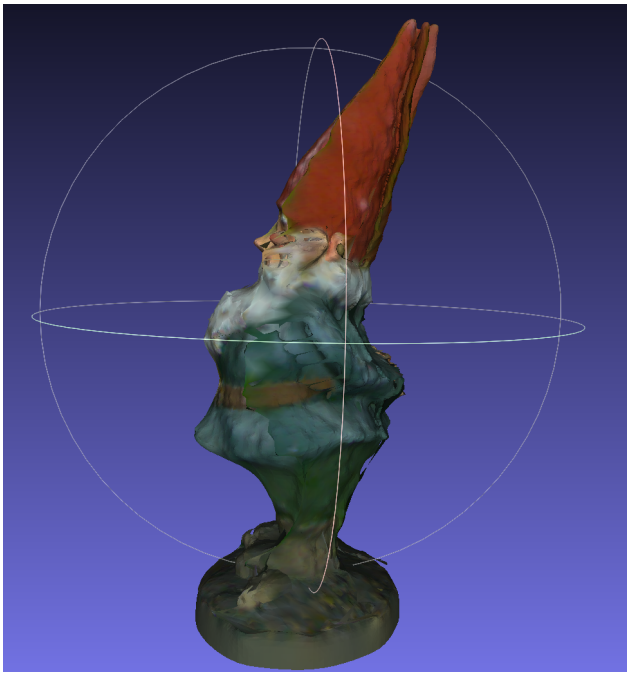


Figure 16: Left Side View

Compared to the original 3D model, the reconstruction still observed some clear similarities. As shown in Figure 13, the belt is still obviously there, along with the large white mustache and the hat. While the hat as well as face are fairly corrupted because of failures in correspondence, there are still obvious features such as the eyes, the blush, and the ears.

Limitations The biggest weakness within the model pipeline was still the manual correspondence point matching. No matter how visually precise I tried to click matching points, there were still small differences that led to nearest point matching to select a different point with a different depth than actually desired. Furthermore, large rotations between scans led to difficulties selecting points that were on the model itself, which limited the ability for me to select accurate corresponding points. In large part, this was the weakest link within the data itself. By creating more scans with smaller rotations between them, correspondence matching would be significantly improved.

Future Modifications In order to create a better correspondence method, one idea is to visualize the 2D points that were correctly reconstructed and matched over the 2D image that the user clicked. This would allow the user to select the precise 2D point that they were targeting, and also prevent inaccuracies, such as selecting the stump instead of the grid below it.

4 Assessment and Evaluation

A Appendix

A.1 calibrate.py

The code written in `calibrate.py` was largely taken from the provided code from **Assignment 3**. It was adapted to be able to handle calibration of two cameras at once, as well as estimating the extrinsic parameters.

A.2 camutils.py

The code written in `camutils.py` was entirely taken from **Assignment 4**, and there was no modification.

A.3 correspond.py

A.3.1 click correspondences

The code was entirely written by myself. I used the matplotlib documentation. This is the actual GUI for selecting the corresponding points.

A.3.2 nearest reconstructed points

The code was entirely written by myself. I used the numpy documentation to help me, along with the guidance provided in the **Project Guidelines**. This is the code for finding the nearest 2D point from the point selected in the image, as well as the distance between those points.

A.3.3 click pairs to 3d

The code was entirely written by myself. It is a wrapper for `nearestreconstructedpoints`, which matches the nearest point for both scan images, and then removes any points that exceed the maximum distance limit.

A.3.4 select 3d correspondences

The code was entirely written by myself. It is a wrapper for both `clickcorrespondences` and `clickpairsto3d`. It first creates the GUI for the user to select points from the images, and then calls `clickpairsto3d` to match the nearest point, and then returns the list of valid correspondence points.

A.3.5 svd alignment

The code was based off the pseudocode provided in **Lecture 12**. It computes the best possible rotation and translation to align the provided correspondence points.

A.3.6 transform points

This code is written by myself, simply to apply the given rotation and translation to a set of input points.

A.3.7 icp point to point

This code was written by myself, with the help of several online sources. First, I used **Lecture 12** as well as the **Project Guidelines** to first understand what ICP was. Then, I followed the ICP Wikipedia Article and the LearnOpenCV Tutorial to understand the pseudocode of the ICP method. I then translated this pseudocode, into the final function.

While initially, I skipped the k-d tree implementation, I later used a SciPy k-d tree implementation to speed up the ICP function, otherwise the program would freeze.

A.4 meshgeneration.py

A.4.1 getcorrespondingpoints

This function is a wrapper behind the reconstruct function, which takes the camera calibration and calculates the 3D point reconstructions.

A.4.2

A.5 meshutils.py

This is entirely copied from the given **Project Guidelines** page. I use this to convert my mesh into .ply format.

B reconstruct.py

B.0.1 decode

This function is entirely copied from **Assignment 4**. It computes the gray code for the object.

B.0.2 reconstruct

This function is almost entirely copied from **Assignment 4**, but I add an additional mask for the object color. Using this, I was able to compute the object's color as well, and remove any background points.

C finalproject.ipynb

This notebook contains the actual code and full process setup for running all of the functions together.